ICS570 Lab 2 — Part B Addendum: AES-GCM Protection of a Student-Data Export

Group 1 members:

Student ID	Name
G35532028	Elise Oyilonye Kennedy-Angbo
G90412028	Joseph Delali Kodzo Etse
G01322028	Liza Ekua Ndede Aikins
G25522028	Eugene Emmanuel Sewor

Run with `npm run demo:crypto` (`scripts/part-b-crypto-demo.js`, log in `evidence/part-b-crypto-log.json`). The script uses Node's built-in `crypto` AES-256-GCM; no AES is implemented by hand.

```
[1] Fresh 96-bit Nonce Verification (Two encryptions of identical record):
    Encryption #1 Nonce (96-bit): 395d0368af7b839ff4b830e5
    Encryption #2 Nonce (96-bit): 950058cc97cf51143cbc8e65
    Nonces Unique?                true
    Ciphertexts Distinct?         true

[2] Authorized AES-256-GCM Decryption:
    Auth Tag (128-bit):           4630ff0797243536d3ddc99857f1399f
    Restored Matches Original?    true
    Restored Student ID & Grade:  S001 -> A

[3] Integrity & Tamper Detection Tests:
    Tampered Ciphertext Byte Rejected? true (Unsupported state or unable to authenticate data)
    Tampered Auth Tag Byte Rejected?   true (Unsupported state or unable to authenticate data)
```

Why Nonce Uniqueness and Key Protection Matter

In AES-GCM, the 96-bit nonce initializes the counter mode keystream (`CTR`) and the Galois field polynomial authenticator (`GHASH`). Reusing a `(key, nonce)` pair across two plaintexts leaks the XOR of both plaintexts (`C1 ⊕ C2 = P1 ⊕ P2`) and allows an attacker to solve for the internal `GHASH` authentication key `H`, enabling arbitrary ciphertext forgery.

Why AES-GCM Does Not Replace Passkey Authentication or Authorization

AES-GCM provides confidentiality and integrity for data in transit or at rest (such as an exported student transcript file), ensuring that unauthorized parties without the secret key cannot read or silently alter the exported ciphertext. However, encryption does not identify who is sitting at the browser or making an HTTP request; WebAuthn passkey authentication is required to cryptographically prove the live identity of the user (e.g., `S001` versus `L001`). Furthermore, even if the application server holds the AES-GCM key to decrypt records in storage, server-side authorization is still required on every request to decide whether the authenticated user has permission to view a specific student record or modify a course grade. Without authentication and per-request authorization, an application that automatically decrypts records for any logged-in session would still allow `S001` to read `S002`'s decrypted record (T6) or overwrite grades (T7).